UNIVERSIDAD NACIONAL DE INGENIERÍA

UNIDAD DE POSGRADO FACULTAD DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN / ESCUELA DE POSGRADO



PLAN DE TESIS

"ALINEACIÓN ENTRE CONTRATOS OPENAPI BANCARIOS Y MODELOS DE REFERENCIA BIAN MEDIANTE ANÁLISIS ESTRUCTURAL Y SIMILITUD SEMÁNTICA"

PARA OBTENER EL GRADO ACADÉMICO DE
MAESTRO EN CIENCIAS CON MENCIÓN EN:
INTELIGENCIA ARTIFICIAL

ELABORADA POR:

ALEX MANCILLA ANTAY
DENNIS JACK PAITAN CANO

Grupo 10

ASESOR:

POR DESIGNAR

LIMA, PERÚ

2026

Datos generales

1. **Título del plan de tesis:** Alineación entre contratos OpenAPI bancarios y modelos de referencia BIAN mediante análisis estructural y similitud semántica
2. **Autor(es):** Alex Mancilla Antay; Dennis Jack Paitan Cano
3. **Asesor(es):** Por designar
4. **Área involucrada:** Inteligencia Artificial — alineación de APIs bancarias OpenAPI con modelos BIAN
   Unidad de Posgrado: Facultad de Ingeniería de Sistemas y Computación
   Programa: Maestría en Inteligencia Artificial
5. **Lugar o institución donde se desarrolla el proyecto:** Universidad Nacional de Ingeniería, Lima, Perú
6. **Duración estimada del proyecto:** 6 meses (Marzo 2026 — Junio 2026)

Resumen

El plan mide, con un score, si un contrato OpenAPI bancario corresponde al *Service Domain* BIAN con el que se lo empareja. El expediente del TSB —cerca de 1,9 millones de clientes sin servicio en 2018 (House of Commons Treasury Committee, 2018)— y la orden que obligó a nueve bancos británicos de *retail* a publicar APIs (CMA, 2016) se citan como síntomas de mover negocio y tecnología sin esa medida. No son el objeto.

Trabajamos sobre el contrato. **SemanticScore (S)** mira nombres y textos; **StructuralScore (E)**, si *paths* y verbos se parecen a *behaviors*; **CoverageScore (C)**, qué parte del catálogo BIAN aparece en el YAML. Los tres entran en **AlignmentScore**. El Anexo C dice qué fila de la matriz corresponde a cada paso.

Esta Guía 1 no corre el parser ni puntúa un piloto. Deja escrito **cómo** se calcularía. Latencia, *fuzzing* y seguridad en vivo no entran (§1.6).

Palabras clave: OpenAPI; BIAN; AlignmentScore; schema matching; API bancaria; alineación estructural.

Abstract

OpenAPI files go live. BIAN Service Domains stay in architecture decks. We have not seen a published score that puts the two artefacts side by side before release.

Group 10 writes that score. Meaning (S), REST shape versus BIAN behaviours (E), and how much of the Service Domain the YAML covers (C) feed AlignmentScore. The consistency matrix is Appendix C, not a slogan in the introduction.

Guide 1 is the design. Parser, bank pilot and runtime security are later work (§1.6).

Keywords: OpenAPI; BIAN; AlignmentScore; schema matching; banking API; structural alignment.

Índice general

Datos generales I

Resumen II

Abstract III

1. Planteamiento del problema 1
- 1.1. Introducción . . . 1
- 1.2. Problemática . . . 2
- 1.3. Planteamiento del problema . . . 3
  - 1.3.1. Situación real (problemática) . . . 3
  - 1.3.2. Situación subyacente (problema abstracto) . . . 3
  - 1.3.3. Problema tecnológico (situación a modificar) . . . 3
  - 1.3.4. Problema de investigación . . . 3
- 1.4. Objeto de estudio . . . 4
  - 1.4.1. Producto central: modelo de alineación . . . 6
- 1.5. Ámbito y delimitación . . . 7
- 1.6. Alcances y limitaciones . . . 7
- 1.7. Justificación e importancia . . . 8
- 1.8. Clasificación del tema (IEEE / ACM) . . . 9
- 1.9. Tipo de investigación . . . 9

2. Objetivos, hipótesis y factores 10
- 2.1. Línea de base / datos iniciales . . . 10
- 2.2. Objetivo superior . . . 10
- 2.3. Objetivo general (técnico) . . . 11
- 2.4. Objetivos específicos . . . 11
- 2.5. Hipótesis . . . 11
- 2.6. Factores y ratios de la investigación . . . 12
- 2.7. Figura de cadena de objetivos . . . 12

3. Marco teórico 14
- 3.1. Antecedentes de la investigación . . . 14
- 3.2. Bases conceptuales . . . 14
  - 3.2.1. API REST y contrato OpenAPI . . . 14
  - 3.2.2. Modelo de referencia BIAN . . . 15
  - 3.2.3. Alineación, coherencia y desalineación . . . 15
  - 3.2.4. Modelo de puntuación . . . 15
- 3.3. Definición de términos (mapa semántico) . . . 15
- 3.4. Bases teóricas . . . 16
  - 3.4.1. Correspondencia estructural (*schema matching*) . . . 16

3.4.2. Alineación ontológica y similitud semántica . . . 17
3.4.3. Representación enriquecida de contratos . . . 17
3.4.4. Relación con el modelo propuesto . . . 17
3.5. Estado del arte por ejes temáticos . . . 17
3.5.1. Eje E1 — OpenAPI como estándar . . . 17
3.5.2. Eje E2 — Matching estructural . . . 18
3.5.3. Eje E3 — Matching semántico y ontologías . . . 18
3.5.4. Eje E4 — Microservicios e interfaces . . . 18
3.5.5. Eje E5 — Testing y calidad API (borde) . . . 18
3.5.6. Eje E6 — Selección y recomendación de servicios . . . 18
3.6. Brecha y oportunidad de investigación . . . 19
3.6.1. Qué sí cubre la literatura revisada . . . 19
3.6.2. Vacíos identificados . . . 20
3.6.3. Oportunidad de investigación . . . 20

4. Diseño metodológico **21**
4.1. Tipo y enfoque metodológico . . . 21
4.2. Procedimiento general . . . 21
4.2.1. Relación objetivos, fases y procedimiento . . . 21
4.2.2. Protocolo de validación empírica (fuera de alcance) . . . 22
4.3. Fuentes de datos . . . 23
4.3.1. Adquisición de datos . . . 23
4.3.2. Criterios de inclusión (muestra empírica) . . . 23
4.4. Técnicas y herramientas . . . 24
4.5. Métricas de evaluación . . . 24
4.6. Casos de estudio . . . 24
4.7. Criterios de validez . . . 24
4.8. Amenazas a la validez . . . 24

5. Administración del plan **26**
5.1. Cronograma . . . 26
5.2. Presupuesto . . . 26
5.3. Financiamiento . . . 27
5.4. Recursos . . . 27
5.4.1. Recursos humanos . . . 27
5.4.2. Recursos materiales y de software . . . 28
5.4.3. Recursos bibliográficos . . . 28
5.5. Roles del equipo . . . 28
5.6. Riesgos . . . 28

A. String de búsqueda y criterios de selección **29**
A.1. Bases de datos consultadas . . . 29
A.2. Período y idioma . . . 29
A.3. Cadenas de búsqueda (inglés) . . . 29
A.4. Criterios de inclusión . . . 29
A.5. Criterios de exclusión . . . 30
A.6. Resultado de la búsqueda . . . 30

B. Matriz de papers bibliográficos **31**
B.1. Resumen por paper . . . 31
B.2. Mapa de ejes . . . 32

C. Matriz de consistencia **33**
C.1. Metadatos . . . 33
C.2. Resumen por fase . . . 33
C.3. Trazabilidad Cap. 2 . . . 34

D. Matriz de operación de variables **35**
D.1. Ratios principales . . . 35
D.2. Factores por objetivo específico . . . 36
D.3. Notas de medición . . . 36

E. Diseño factorial (si aplica) **37**

F. Ejemplos OpenAPI y BIAN **38**
F.1. Ejemplo reducido (unidad de análisis) . . . 38
F.2. Ejemplo ampliado (fragmento de contrato) . . . 38
F.2.1. Contrato OpenAPI (extracto ilustrativo) . . . 38
F.2.2. Referencia BIAN (Payment Initiation — ilustrativo) . . . 39
F.2.3. Correspondencias para el modelo S/E/C . . . 39
F.3. Escenarios numéricos ilustrativos . . . 40
F.4. Nota metodológica . . . 40

Referencias bibliográficas **41**

Anexo de entrega — repositorio digital **44**

Índice de figuras

1.1. Contexto: arquitectura API bancaria, BIAN, OpenAPI y modelo S/E/C 2
1.2. Mapa conceptual del estudio: objeto, referencia, unidad de análisis y producto del modelo (elaboración propia). 5
1.3. Objeto de estudio, unidad de análisis y producto (ejemplo genérico) 6
1.4. Cálculo del score de alineación: S, E, C → **AlignmentScore** 7

2.1. Cadena de objetivos: normalización → scores parciales → score global. Fase 1 (diseño del modelo) precede a OE1; ver Anexo C. 13

3.1. Bases conceptuales: contrato OpenAPI, *Service Domain* BIAN y modelo de alineación S/E/C 15
3.2. De la literatura revisada (E1–E6) a la brecha de investigación (V1–V5) 20

4.7. Correspondencia jerárquica para el cálculo de **StructuralScore (E)**. Fuente: elaboración propia; ejemplo ilustrativo. 22

Índice de tablas

1.1. Preguntas específicas de investigación 4
1.2. Tipología de desalineaciones OpenAPI–BIAN 4
1.3. Objeto, unidad de análisis y producto del modelo 5
1.4. Ejemplo genérico ilustrativo (sin caso piloto fijo) 5
1.5. Componentes del modelo de alineación 6
1.6. Ámbito y delimitación del estudio 7
1.7. Alcances y limitaciones del estudio 8
1.8. Clasificación IEEE / ACM del tema de investigación 9

2.1. Objetivos específicos como transformaciones 11
2.2. Factores y ratios por objetivo específico 12

3.1. Mapa semántico — términos clave del plan de tesis 16
3.2. Síntesis del estado del arte por ejes (E1–E6) 19
3.3. Vacíos identificados en la literatura (V1–V5) 20

4.1. Objetivos específicos vinculados al procedimiento metodológico 22
4.2. Adquisición de datos — diseño metodológico 23

5.1. Cronograma de elaboración del plan de tesis 26
5.2. Presupuesto estimado del plan de tesis 27
5.3. Distribución de roles del equipo 28
5.4. Matriz de riesgos del plan de tesis 28

B.1. Resumen de papers del estado del arte (Anexo B) 31
B.2. Ejes temáticos del estado del arte 32

C.1. Matriz de consistencia — resumen por fase (Anexo C) 33
C.2. Vinculación fases del modelo $\rightarrow$ objetivos específicos 34

D.1. Operación de variables de investigación (Anexo D) 35
D.2. Operacionalización de factores 36

F.1. Ejemplo reducido — pares OpenAPI $\leftrightarrow$ BIAN 38
F.2. Elementos BIAN emparejables con el extracto OpenAPI 39
F.3. Lectura del ejemplo bajo S, E y C 39
F.4. Escenarios ilustrativos — S, E, C $\rightarrow$ **AlignmentScore** 40

Capítulo 1

Planteamiento del problema

El grupo parte de un YAML bancario y de un *Service Domain* BIAN, no de un banco con nombre. Lo que sigue es el dolor del sector, los tres niveles que pidió el curso (PI-11), el objeto y hasta dónde llega Guía 1.

1.1. Introducción

El plan arranca de una decisión del grupo: **medir** si un contrato OpenAPI bancario y un *Service Domain* BIAN emparejado hablan del mismo negocio. Un número, no una reunión de arquitectura que termina en “parece alineado”.

OpenAPI es el archivo YAML o JSON con *paths*, verbos HTTP y *schemas* (Casas et al., 2021; Santos et al., 2020). BIAN —*Banking Industry Architecture Network*— parte el negocio en *Service Domain*s (pagos, cuentas) con *behaviors* y *business objects* (Farzi, 2021). Un equipo publica el contrato; otro conserva el modelo de referencia. Casi nunca hay una métrica que los cruce **antes** del despliegue.

Llamamos **alineación** a ese cruce. El producto es un modelo de puntuación: **SemanticScore (S)**, **StructuralScore (E)**, **CoverageScore (C)** y **AlignmentScore** $= \alpha S + \beta E + \gamma C$ con $\alpha + \beta + \gamma = 1$ (§1.4 y Fig. 1.4).

El sector ya mostró el costo de no hacerlo. TSB (2018) y el *open banking* británico (CMA, 2016) aparecen en §1.2 como síntomas; el objeto —el contrato, no el banco— se fija en §1.4; el problema tecnológico, en §1.3.

Guía 1 **diseña** el procedimiento y lo deja en el Anexo C. El parser y los scores se corren en Tesis 2 (§1.6).

La Figura 1.1 pone las piezas en una sola vista: APIs, BIAN, el YAML de entrada y S/E/C de salida.

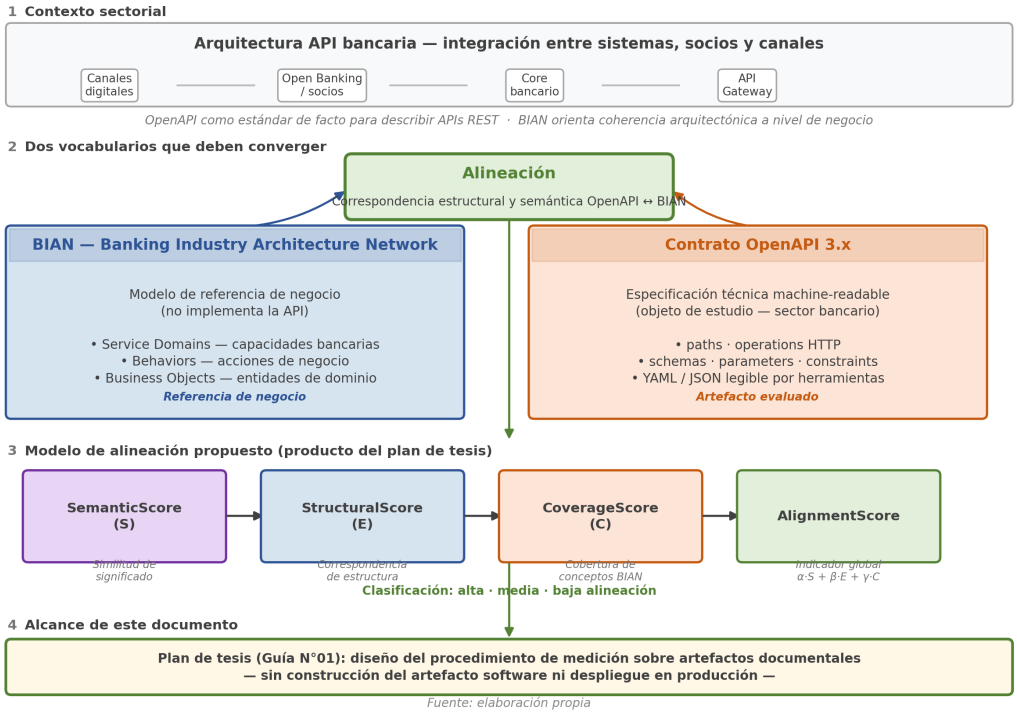


Figura 1.1: Contexto: arquitectura API bancaria, BIAN, OpenAPI y modelo S/E/C

TSB y CMA van en §1.2. El objeto, en §1.4. Objetivos, marco y método ocupan lo que resta del documento.

1.2. Problemática

En abril de 2018 el TSB migró la plataforma y cerca de 1,9 millones de clientes se quedaron sin servicio. El informe de la Cámara de los Comunes cifra el episodio en cientos de millones de libras y en semanas de operación restringida (House of Commons Treasury Committee, 2018). No habla de OpenAPI ni de BIAN. Documenta otra cosa: el costo de cambiar el sistema cuando negocio y tecnología no tienen una correspondencia que se pueda revisar. Retrabajo, adaptadores armados a prueba y error, daño de reputación.

El *open banking* británico es un caso distinto. La orden de la CMA (2016) obligó, a partir de 2018, a nueve bancos de *retail* a exponer APIs. Publicar el Swagger no dice cómo se compara esa interfaz con un modelo de negocio de referencia. Esa comparación sigue en la gobernanza del banco, no en el archivo.

En lo que leímos, el hueco se repite con otros números. Casas et al. (2021) mapean 47 estudios primarios y solo el 40 % ancla un dominio; el resto trabaja APIs genéricas. Golmohammadi et al. (2024) revisan 92 trabajos de prueba REST: validar un servicio ya desplegado sale caro, y los casos industriales que se pueden repetir son pocos. Farzi (2021) acerca BIAN a la agilidad del banco y deja abierta la distancia entre el *to-be* y lo que corre.

En un contrato el desajuste no es un ensayo. Arquitectura guarda el *Service Domain*; desarrollo versiona el YAML del *sprint*. Para la misma orden aparecen `PaymentOrder` y `paymentRequest`. Un

behavior declarado con PUT no tiene el GET en el contrato, o al revés, y la versión nueva del archivo no dispara una revisión del modelo. Cuando hay revisión, es lectura humana o un *schema matcher* que no conoce el SD. Shabbir et al. (2025) atan ese desfase a la consistencia y a la mantenibilidad de la documentación. El §1.3 lo formula como problema tecnológico; el Anexo C, como fila de la matriz.

1.3. Planteamiento del problema

Hacen falta tres cortes. Si los tres cuentan el TSB, sobran dos.

1.3.1. Situación real (problemática)

Los casos y las cifras quedan en §1.2: el TSB, la orden de la CMA, el 40 % de Casas et al. (2021) y la revisión de Golmohammadi et al. (2024). Ahí hay clientes fuera de servicio y APIs exigidas por regulación. No hay un contrato elegido ni una fila de la matriz.

1.3.2. Situación subyacente (problema abstracto)

Sin el nombre del banco quedan dos estructuras que no coinciden de antemano. El contrato es un árbol de *paths*, operaciones y *schemas*. El *Service Domain* es otro: *behavior*s, *business object*s y relaciones de control. El problema abstracto es esa correspondencia. Los tipos —nombre, verbo HTTP, objeto no cubierto, versión del YAML que no revisa el modelo— están en la Tabla 1.2. El par `PaymentOrder` / `paymentRequest` ya se usó en §1.2; aquí no se repite.

De ese corte salen el objeto y la unidad de §1.4: el contrato completo, y cada par endpoint/schema $\leftrightarrow$ *behavior*/*business object*. BIAN entra como documento de referencia, no como el *core* del banco.

1.3.3. Problema tecnológico (situación a modificar)

Lo que un equipo tiene hoy es el Swagger leído a ojo, o un *schema matcher* que no distingue *Service Domain*s. Para el par contrato $\times$ SD no hay un **StructuralScore (E)**, un **SemanticScore (S)**, un **CoverageScore (C)** ni un nivel (Alta, Media, Baja, Nula) con los umbrales escritos. El objetivo general del Capítulo 2 y la matriz del Anexo C describen ese procedimiento. Correrlo sobre el *core* de un banco queda fuera de esta Guía (Apartado 1.6).

1.3.4. Problema de investigación

El **problema de investigación** es una pregunta, no un objetivo:

> **¿En qué medida, y con qué tipos de desajuste, un contrato OpenAPI bancario se aparta del *Service Domain* BIAN emparejado, y cómo se convierte eso en un score?**

El §3.6 dice qué parte de esa pregunta dejan abierta los 14 papers.

Las **preguntas específicas** bajan el mismo hueco a cuatro cortes, no a cuatro objetivos:

Tabla 1.1: Preguntas específicas de investigación

ID	Pregunta
P1	¿Qué correspondencias estructurales pueden establecerse entre elementos de un contrato OpenAPI (*paths*, *operations*, *schemas*) y los artefactos de un *Service Domain* BIAN?
P2	¿Qué grado de similitud semántica existe entre los términos y conceptos de negocio en BIAN y los nombres de recursos, propiedades y descripciones en OpenAPI?
P3	¿Qué tipologías de desalineación predominan al comparar ambos artefactos en un mismo dominio de servicio?
P4	¿Es posible integrar métricas estructurales y semánticas en un indicador de coherencia global por cada instancia evaluada (par contrato OpenAPI bancario — *Service Domain* BIAN)?

La Tabla 1.2 resume la tipología de desalineaciones identificadas:

Tabla 1.2: Tipología de desalineaciones OpenAPI–BIAN

Tipo	Descripción	Ejemplo ilustrativo
Estructural	Divergencia entre la organización de endpoints/esquemas y la estructura del SD BIAN	Operación `POST /transfer` sin correspondencia con el *behavior* `Initiate` definido en BIAN
Semántica	Equivalencia estructural parcial pero significado de negocio distinto	Propiedad `amount` sin vínculo explícito con `PaymentAmount` del modelo BIAN
De modelado	Representación incorrecta o incompleta de entidades de negocio	Esquema que omite atributos obligatorios del *business object* BIAN
REST / contrato	Inconsistencias en el estilo REST que dificultan la interpretación alineada al dominio	Uso de verbos en URLs o recursos que no mapean a *Service Operations* BIAN

P1–P4 no son los objetivos. Normalizar, emparejar y puntuar están en OE1–OE4 (Capítulo 2) y en las cinco fases del Anexo C.

1.4. Objeto de estudio

El **objeto** es el YAML (o JSON) OpenAPI 3.x con el que un banco, un integrador o un catálogo describe la API. Se puede parsear. Se puede poner al lado de un *Service Domain*. No es el banco, no es BIAN entero y no es el *backend*.

BIAN entra como **referencia**: *behaviors*, *business objects*, control. El consorcio también publica OpenAPI de muestra; esos YAML no son el objeto. El objeto es el contrato **bancario** que se evalúa. El Anexo F lo muestra con *Payment Initiation* sin convertirlo en el único caso.

Tres niveles, porque si se mezclan el score no se interpreta. La Figura 1.2 y la Tabla 1.3 los separan.

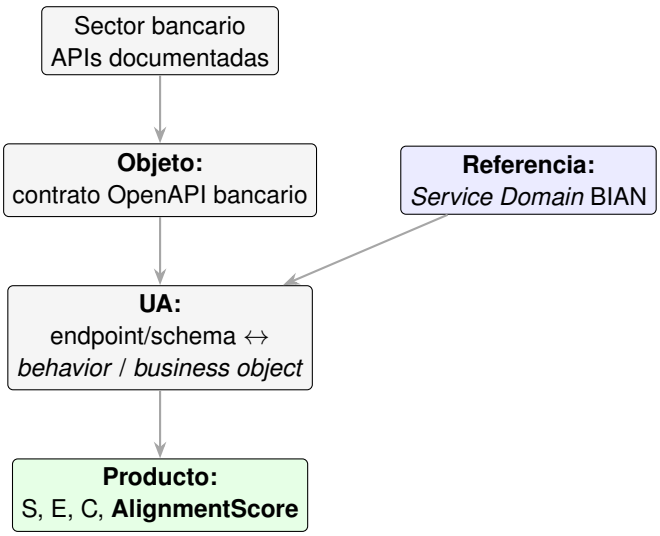


Figura 1.2: Mapa conceptual del estudio: objeto, referencia, unidad de análisis y producto del modelo (elaboración propia).

Tabla 1.3: Objeto, unidad de análisis y producto del modelo

Concepto	**En este estudio**
Objeto de estudio	El contrato OpenAPI bancario completo (artefacto documental)
Unidad de análisis	Elementos comparables dentro del *pipeline*: endpoint/operation/schema ↔ *behavior* / *business object* BIAN
Producto del modelo	Scores S, E, C y **AlignmentScore** por contrato evaluado

El archivo es el objeto. Cada par endpoint–*behavior* o esquema–*business object* es la unidad. El **AlignmentScore** y el nivel (Alta/Media/Baja/Nula) se reportan por instancia `contrato` × *Service Domain*.

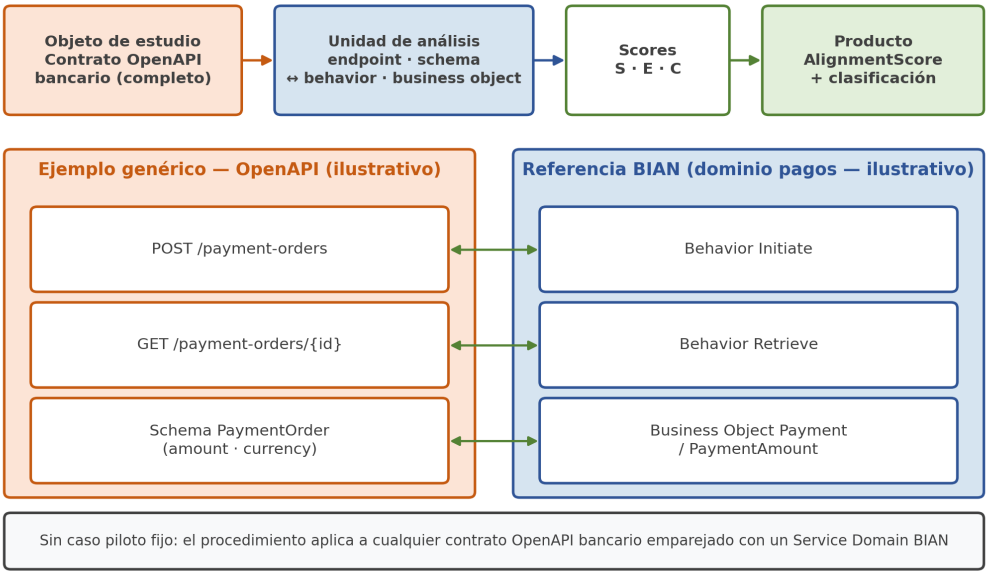
Cada instancia se identifica de forma unívoca, p. ej. `{fuente}_{service-domain}_{version}`, coherente con la adquisición de datos del Capítulo 4 (§4.3).

Tabla 1.4: Ejemplo genérico ilustrativo (sin caso piloto fijo)

Elemento OpenAPI (genérico)	**Elemento BIAN de referencia (ilustrativo)**
`POST /payment-orders`	*behavior* `Initiate` del dominio de pagos
`GET /payment-orders/{id}`	*behavior* `Retrieve`
Esquema `PaymentOrder` (propiedades `amount`, `currency`)	*business object* `Payment` / `PaymentAmount`

La tabla es **explicativa**: nombres ficticios para mostrar la unidad de análisis. El caso ampliado de *Payment Initiation* está en Anexo F; el procedimiento aplica a cualquier contrato OpenAPI bancario emparejado con un *Service Domain* BIAN. La Figura 1.3 resume objeto, unidad de análisis, producto del modelo y el ejemplo genérico.

Figura 2. Objeto de estudio, unidad de análisis y producto del modelo



Fuente: elaboración propia — coherente con Tabla 3 §1.4

Figura 1.3: Objeto de estudio, unidad de análisis y producto (ejemplo genérico)

1.4.1. Producto central: modelo de alineación

Lo que la matriz pide como producto es el **modelo**, no un dashboard. Entrada: un contrato y un SD emparejado. Salida:

Tabla 1.5: Componentes del modelo de alineación

Componente	Significado
SemanticScore (S)	Similitud de significado entre conceptos OpenAPI y BIAN
StructuralScore (E)	Correspondencia de jerarquía y organización (REST ↔ SD)
CoverageScore (C)	Proporción de conceptos BIAN cubiertos por el contrato

A partir de estos componentes se obtiene el **AlignmentScore**:

$$\textbf{AlignmentScore} = \alpha \cdot S + \beta \cdot E + \gamma \cdot C, \text{ con } \alpha + \beta + \gamma = 1$$

La clasificación interpretativa del *score* es: **Alta** ($\geq 0{,}80$), **Media** (0,60–0,79), **Baja** (0,40–0,59) y **Nula** ($< 0{,}40$). Los umbrales son **convención provisional del diseño** para este plan; su calibración empírica permanece fuera del alcance (§1.6). Los scores S, E y C se calculan por unidad de análisis (par UA); el **AlignmentScore** y la clasificación se reportan por instancia contrato $\times$ *Service Domain*. La Figura 1.4 ilustra el cálculo del score de alineación.

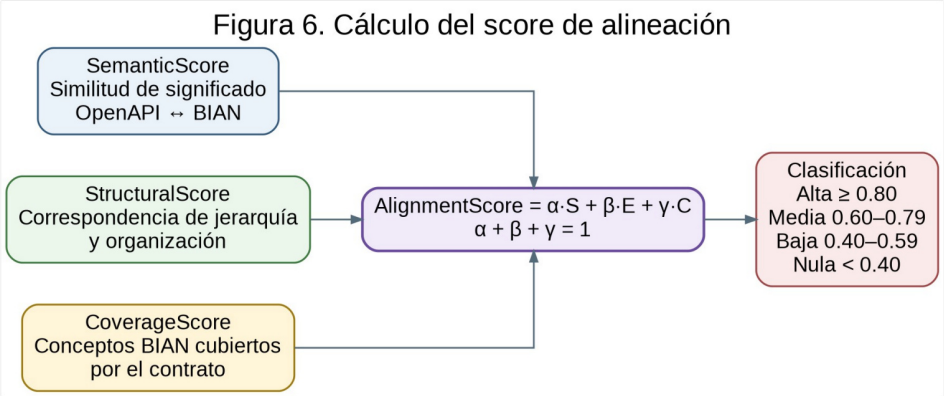


Figura 1.4: Cálculo del score de alineación: S, E, C → **AlignmentScore**

Diseño, normalización, E, S, C y el score integrado: eso es Anexo C y OE1–OE4 (Capítulo 2).

1.5. Ámbito y delimitación

Tabla 1.6: Ámbito y delimitación del estudio

Dimensión	Delimitación
Sector	Industria bancaria y servicios financieros que adoptan arquitecturas API
Artefactos	Contratos OpenAPI 3.x y especificaciones BIAN (*Service Domain* en formato publicado por BIAN)
Técnica	Análisis estructural y similitud semántica (*matching*, *embeddings*, métricas de proximidad)
Alcance geográfico	No restringido; casos de referencia con documentación pública internacional
Temporal	Especificaciones vigentes al momento del estudio (2024–2026)

Fuera: escribir o desplegar la API, penetración, *fuzzing*, latencia y disponibilidad en *runtime*.

1.6. Alcances y limitaciones

Esta Guía deja escrito el modelo y el procedimiento de cálculo. No incluye el software ni una corrida sobre decenas de contratos. La Tabla 1.7 separa lo que entra de lo que queda fuera.

Tabla 1.7: Alcances y limitaciones del estudio

Alcance	**Limitación**
Evaluar alineación **estructural** y **semántica** entre contratos OpenAPI y modelos BIAN	Despliegue de APIs en producción
Definir **métricas reproducibles** aplicables sobre artefactos documentales	Certificación regulatoria y auditoría legal
Definir un **modelo de alineación** (S, E, C $\rightarrow$ **AlignmentScore**) aplicable a contratos OpenAPI bancarios	Catálogo completo de *Service Domain*s BIAN
Analizar contratos OpenAPI del sector bancario frente a *Service Domain*s BIAN de referencia	Encuestas o percepción de desarrolladores
Integrar hallazgos en **AlignmentScore** y clasificación por niveles de alineación	Testing de APIs como foco (Ehsan et al., 2022; Golmohammadi et al., 2024)
Documentar el **diseño** del modelo de alineación y del procedimiento de cálculo	Construcción del software y experimento a escala

1.7. Justificación e importancia

Farzi (2021) y Bhatia (2022) describen la deriva entre producto, integración y gobernanza cuando el contrato y BIAN dejan de rastrearse. Un *release* puede alejarse del modelo de negocio sin que eso se mida hasta producción. Con decenas de APIs y de socios, leer el Swagger a mano no ordena la cola de revisión.

AlignmentScore, junto con P1–P4 (§1.3), pone un número sobre el YAML. Shabbir et al. (2025) piden exactamente eso para consistencia y mantenibilidad: evidencia, no el parecer de una reunión de arquitectura. Casas et al. (2021) ya dejan OpenAPI maduro como artefacto. Lo que este plan agrega es el cruce con un *Service Domain* y un procedimiento que otro pueda repetir (Anexo C).

S y E salen de *schema matching* y de similitud semántica. No reemplazan pruebas de *runtime* ni una certificación (eje E5, §3.5). La pregunta, en la práctica, es si esta versión del contrato refleja este SD.

1.8. Clasificación del tema (IEEE / ACM)

Tabla 1.8: Clasificación IEEE / ACM del tema de investigación

Taxonomía	Categoría	Justificación
IEEE	*Knowledge Representation*	Los contratos OpenAPI y los modelos BIAN son representaciones formales de conocimiento de dominio; el estudio evalúa correspondencia y coherencia entre representaciones
ACM	*Semantic networks* (y ontologías / alineación de esquemas)	El *matching* estructural y la similitud semántica entre conceptos de negocio y componentes API se enmarcan en redes de significado y alineación ontológica

1.9. Tipo de investigación

Es *design science* sobre artefactos, no sobre personas. El modelo toma un YAML y un extracto BIAN y debe poder devolver S, E, C y **AlignmentScore**. No hay encuesta, no hay muestra humana, no hay experimento de laboratorio con sujetos.

La validez se juega en si otro grupo puede seguir el Anexo C y llegar al mismo tipo de producto, y en las amenazas del Capítulo 4. No en un p-valor sobre entrevistas.

Capítulo 2

Objetivos, hipótesis y factores

Datos de entrada, lo que se promete y lo que se hipotetiza. La figura del cierre ata OE1–OE4 al Capítulo 4.

2.1. Línea de base / datos iniciales

No hay mediciones previas del grupo. La línea de base son los archivos con los que trabajaría el modelo:

- **Entrada principal:** contratos OpenAPI 3.x del sector bancario (objeto de estudio, Cap. 1 §1.4).
- **Referencia de contraste:** especificaciones de *Service Domain* BIAN (*behavior*, *business object*, relaciones de control).
- **Producto esperado:** representaciones normalizadas, scores S/E/C y **AlignmentScore** con clasificación por niveles.

Quedan fuera las implementaciones en producción, el tráfico *runtime* y el testing automatizado (Apartado 1.6). La Tabla 1.4 y el Anexo F muestran el formato del cálculo. No eligen el piloto: entra cualquier par contrato OpenAPI bancario — *Service Domain* BIAN que cumpla los criterios de inclusión del plan.

2.2. Objetivo superior

El objetivo superior describe el uso en un banco, si el modelo llegara a usarse. No es el objetivo técnico de Guía 1.

> Que un equipo de arquitectura pueda medir, sobre el contrato y antes del despliegue, cuánto coincide ese OpenAPI con el modelo BIAN de referencia. El resultado esperado es menos adaptadores rehechos a ciegas y menos deriva entre la API publicada y el negocio que BIAN describe.

Guía 1 no instala ese procedimiento en un banco (Apartado 1.6). Deja el modelo y los pasos. El software y un piloto a escala son otra entrega.

2.3. Objetivo general (técnico)

Diseñar cómo se calculan S, E, C y el **AlignmentScore** de un contrato OpenAPI frente a un *Service Domain* BIAN, y dejarlo escrito de modo que otro pueda seguir los mismos pasos (Anexo C).

La unidad de medición es el par `contrato` $\times$ `Service Domain`. Sale un **AlignmentScore** en $[0, 1]$ y un nivel. Esa es la variable dependiente. S, E, C y la completitud de OE1 son insumos, no el resultado que cierra la tesis.

OE1 prepara los artefactos. OE2 responde P1 (E). OE3 responde P2 (S). OE4 junta C, el score y las tipologías (P3–P4). Tabla 2.1, Figura 2.1 y Anexo C.

2.4. Objetivos específicos

OE1–OE4 son transformaciones, no deseos. Encajan en las cinco fases del modelo (§4.2.1).

Tabla 2.1: Objetivos específicos como transformaciones

ID	Subproblema	Entrada	Salida (producto)
OE1	Adquisición y normalización	Contrato OpenAPI bancario + *Service Domain* BIAN	Representaciones normalizadas comparables (*paths*, *schemas*, *behavior*, *business object*)
OE2	Alineación estructural (P1)	Representaciones normalizadas	Matriz de correspondencias + **StructuralScore (E)** $\in [0, 1]$
OE3	Alineación semántica (P2)	Representaciones normalizadas	Matriz de similitud + **SemanticScore (S)** $\in [0, 1]$
OE4	Evaluación integrada (P3–P4)	**StructuralScore (E)**, **SemanticScore (S)** y representaciones normalizadas (OE1)	**CoverageScore (C)**; **AlignmentScore**; clasificación (Alta/Media/Baja/Nula); tipologías

La **Fase 1** del modelo (diseño del procedimiento y definición de scores) corresponde al marco metodológico general del plan y no se confunde con OE1–OE4, que son transformadores sucesivos sobre los artefactos.

2.5. Hipótesis

Una por OE. El factor y el ratio de cada una están en la Tabla 2.2; aquí va el contenido.

1. **H1 (OE1).** Si OpenAPI y BIAN se normalizan al mismo modelo intermedio, S, E y C se pueden calcular sobre esas tablas (completitud de la extracción como ratio).

2. **H2 (OE2).** Jaccard de nombres, path y verbo HTTP —la caja de *schema matching* de Shvaiko y Euzenat (2005)— alcanza para un E que distinga un par razonable de uno forzado.

3. **H3 (OE3).** Similitud de texto (El Bayadh et al., 2015; Chandrasekaran y Mago, 2022), y más adelante embeddings, separan un concepto alineado de un falso amigo de modelado. Eso es S.

4. **H4 (OE4).** Con $\alpha = \beta = \gamma = 1/3$ el **AlignmentScore** se puede leer como Alta/Media/Baja/Nula. Cambiar los pesos (sensibilidad) no debe invertir el relato de una instancia, solo el número.

La Fase 1, la del diseño, asume que partir en S, E y C es mejor que un único "índice de coherencia" opaco. El Anexo C lo deja en filas.

2.6. Factores y ratios de la investigación

Los **factores** son las condiciones de diseño cuya variación puede contrastarse en una validación empírica posterior; los **ratios** son los resultados numéricos medidos (scores y clasificación). Los **parámetros de control** mantienen constantes otras condiciones para comparar el efecto de cada factor.

Tabla 2.2: Factores y ratios por objetivo específico

OE	Factores	Ratios (resultados)	Parámetros de control
Artefacto (OG)	Modelo S/E/C; pesos α, β, γ	**AlignmentScore + clasificación** (VD global del artefacto)	Instancia `contrato` $\times$ `SD`; $\alpha+\beta+\gamma = 1$; umbrales §1.4
OE1	Esquema de representación intermedia	Completitud de extracción (elementos extraídos / esperados)	Versión fija del contrato OpenAPI y del *Service Domain* BIAN; herramientas de *parsing* fijas
OE2	Estrategia de *schema matching*	**StructuralScore (E)** $\in [0, 1]$	Mismo conjunto normalizado de OE1; umbral de correspondencia fijo
OE3	Técnica de similitud semántica (embeddings, ontología, híbrido)	**SemanticScore (S)** $\in [0, 1]$	Mismo corpus de términos; umbral de similitud fijo
OE4	Pesos de agregación (α, β, γ)	**CoverageScore (C)** (ratio parcial); contribuye a **AlignmentScore**	$\alpha + \beta + \gamma = 1$ fijos; mismos S, E, C; taxonomía Tabla 1.2

Los indicadores y métricas detallados por fase se documentan en la matriz de consistencia (Anexo C).

2.7. Figura de cadena de objetivos

La Figura 2.1 cierra este capítulo: secuencia de transformaciones desde los artefactos de entrada hasta el **AlignmentScore** integrado.

Figura 4 — Cadena de objetivos: transformaciones entrada → salida

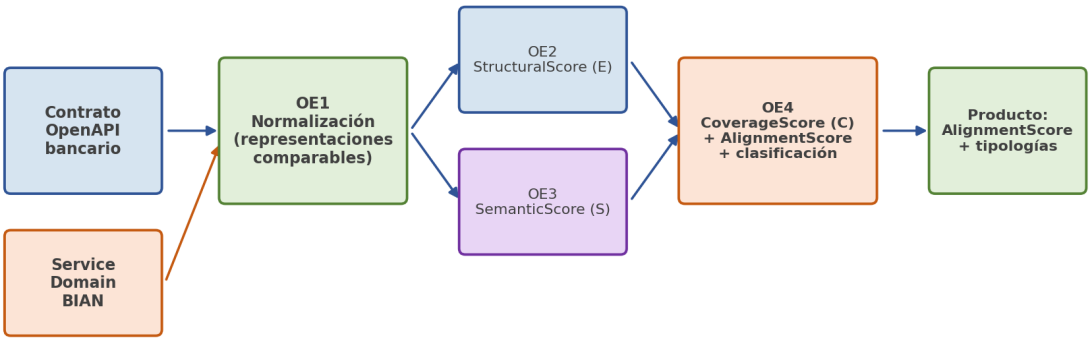


Figura 2.1: Cadena de objetivos: normalización → scores parciales → score global. Fase 1 (diseño del modelo) precede a OE1; ver Anexo C.

Capítulo 3

Marco teórico

Fichas del Anexo B, vocabulario del grupo y recorte V1–V5. El objeto sigue siendo el del §1.4.

3.1. Antecedentes de la investigación

Catorce trabajos, seis ejes. Las fichas están en el Anexo B; aquí solo el recorte que usa el plan. El comentario por paper está en §3.5.

1. **Madurez de OpenAPI.** En 47 artículos (2011–2020), Casas et al. (2021) tratan OpenAPI/Swagger como el estándar de facto de las APIs REST: generación, pruebas y gobernanza. Coherencia con un modelo de negocio bancario no entra en esa revisión.
2. **Correspondencia.** Shvaiko y Euzenat (2005) ordenan el *schema matching*. El Bayadh et al. (2015), Chandrasekaran y Mago (2022) y Santos Sousa et al. (2026) cubren alineación ontológica y similitud. Llegan por separado. El score único del par contrato–SD se define en este plan, no en esos papers.
3. **Dominio e interfaz.** Schmidt y Thiry (2020) revisan 27 estudios de microservicios con MDE y DDD. Sirve para no tratar el YAML como si fuera el *Service Domain*.
4. **Lo que no es el foco.** Testing, seguridad y selección de servicios (E5 y E6) usan OpenAPI para cobertura, vulnerabilidades o ranking por QoS. Eso se excluye en Apartado 1.6.

3.2. Bases conceptuales

Los términos de abajo se usan siempre con el mismo sentido. La tabla está en §3.3. Cómo se calculan, en §1.4.

3.2.1. API REST y contrato OpenAPI

El archivo que entra al procedimiento es YAML o JSON OpenAPI: *paths*, verbos y *schemas*. El microservicio que lo implementa no se recibe y no se puntúa. REST —recurso, HTTP, JSON— nombra el estilo de la interfaz. Casas et al. (2021) ya fijaron OpenAPI como estándar de facto; esta sección no vuelve a demostrarlo.

3.2.2. Modelo de referencia BIAN

BIAN divide el banco en *Service Domain*s. Cada dominio publica *behavior*s, *business object*s y relaciones de control (Farzi, 2021). En el plan el SD es la referencia contra la que se compara el contrato. No se evalúa el código del *core* ni el despliegue en un proveedor.

3.2.3. Alineación, coherencia y desalineación

Alineación es la correspondencia entre el YAML y el SD, medida por **AlignmentScore**. Alta o Media se leen como coherencia operativa del par documental. No equivalen a un sello regulatorio. **Desalineación** cubre el resto de la Tabla 1.2: estructural, semántica, de modelado o de REST/contrato.

3.2.4. Modelo de puntuación

S, E y C se agregan con pesos α, β y γ. La Figura 3.1 deja el contrato a un lado, el SD al otro y el score en el medio.

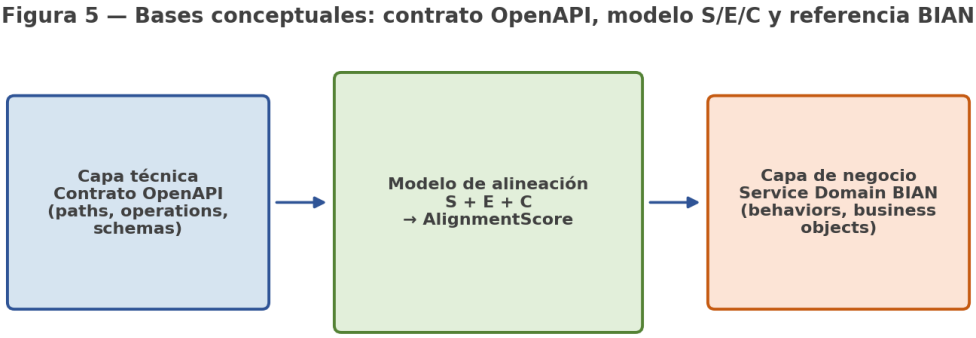


Figura 3.1: Bases conceptuales: contrato OpenAPI, *Service Domain* BIAN y modelo de alineación S/E/C

3.3. Definición de términos (mapa semántico)

El mapa semántico unifica términos empleados en los capítulos del plan. Cuando un concepto aparece en varios capítulos, conserva el significado aquí definido.

Tabla 3.1: Mapa semántico — términos clave del plan de tesis

Término	**Definición en este plan**
API REST	Interfaz que expone recursos y operaciones HTTP con representaciones intercambiables (p. ej. JSON).
Contrato OpenAPI	Documento YAML/JSON (OpenAPI 3.x) que describe *paths*, *operations* y *schemas* de una API.
BIAN	Estándar de arquitectura bancaria que organiza capacidades en *Service Domain*s de referencia.
Service Domain	Unidad de negocio BIAN descrita por *behavior*s, *business object*s y relaciones de control.
behavior	Acción o capacidad de negocio definida en un *Service Domain* BIAN.
business object	Entidad de dominio BIAN (p. ej. orden de pago, cuenta) referenciada en el modelo.
Objeto de estudio	Contrato OpenAPI bancario completo evaluado frente a un *Service Domain* BIAN.
Unidad de análisis	Par comparable endpoint/operation/schema $\leftrightarrow$ *behavior* / *business object*.
Alineación	Grado de correspondencia estructural y semántica entre contrato OpenAPI y *Service Domain* BIAN.
Desalineación	Divergencia detectable; tipologías: estructural, semántica, modelado, REST/contrato (Tabla 1.2).
Representación normalizada	Vista intermedia común (tabular, grafo u ontología) de artefactos OpenAPI y BIAN tras OE1.
Schema matching	Técnicas de correspondencia entre esquemas o estructuras; base del **StructuralScore (E)**.
Similitud semántica	Medida de equivalencia de significado entre términos/conceptos; base del **SemanticScore (S)**.
SemanticScore (S)	Score $\in [0, 1]$ de alineación semántica OpenAPI $\leftrightarrow$ BIAN.
StructuralScore (E)	Score $\in [0, 1]$ de correspondencia estructural REST $\leftrightarrow$ organización del SD.
CoverageScore (C)	Score $\in [0, 1]$ de conceptos BIAN cubiertos por el contrato OpenAPI.
AlignmentScore	Agregación $\alpha S + \beta E + \gamma C$; clasificación Alta/Media/Baja/Nula.
Matching	Procedimiento que establece correspondencias entre elementos de dos artefactos normalizados.

Los términos **SemanticScore (S)**, **StructuralScore (E)**, **CoverageScore (C)** y **AlignmentScore** se usan con la convención **S**, **E**, **C** del modelo (véase §1.4 y Anexo C). El **StructuralScore (E)** mide correspondencia REST $\leftrightarrow$ organización del *Service Domain*; no debe confundirse con similitud ontológica genérica ni con métricas de *runtime*.

3.4. Bases teóricas

Estas bases no repiten el comentario de §3.5. Dejan, de esos papers, solo lo que entra en el cálculo de S, E o C.

3.4.1. Correspondencia estructural (*schema matching*)

Shvaiko y Euzenat (2005) arman la taxonomía en tres cortes que usamos tal cual: *matching* aproximado o exacto; evidencia sintáctica, semántica o externa; grano de elemento o de estructura, más el grado de automatización. El artículo es anterior a OpenAPI y a JSON Schema. Igual fija qué

se compara en el **StructuralScore (E)**: *paths*, *operations* y *schemas* contra *behavior*s y *business object*s del *Service Domain*.

3.4.2. Alineación ontológica y similitud semántica

El Bayadh et al. (2015) agrupan la alineación ontológica en medidas terminológicas, estructurales, extensionales y semánticas, y muestran que una medida sola no alcanza. Por eso los pesos del modelo S/E/C quedan escritos en el plan, no heredados de un solo paper.

Chandrasekaran y Mago (2022) pasan revista a la similitud semántica desde WordNet hasta *embeddings* y *transformers*. Santos Sousa et al. (2026) revisan *embeddings* en *ontology matching* y marcan supervisión, escalabilidad y correspondencias complejas como límites. De ahí salen candidatos para el **SemanticScore (S)** y, con menos fuerza, para el **CoverageScore (C)**. El vocabulario sale del contrato y del SD ya normalizados, no de un corpus general.

3.4.3. Representación enriquecida de contratos

Mainas et al. (2023), ya citados en el eje E1, proponen anotaciones (`x-refersTo`, `x-kindOf`, entre otras), una ontología REST/Hydra y la traducción OpenAPI $\rightarrow$ RDF.

Sobre más de 300 contratos de SwaggerHub, la traducción pasa de 1 s cuando el YAML supera unas 200 líneas. En el Capítulo 4 entran como opción de normalización (OE1). No definen el **SemanticScore (S)**.

3.4.4. Relación con el modelo propuesto

E2 aporta el corte estructural y E3, los candidatos de S y de C. El procedimiento que los junta, con umbrales, está en el Capítulo 4 y en el Anexo C. Por qué hace falta juntarlos: §3.6.

3.5. Estado del arte por ejes temáticos

Seis ejes (E1–E6), los del Anexo B. No responden la misma pregunta. E1 mira si OpenAPI ya es el formato con el que se describe REST. E2 y E3 dicen de dónde pueden salir E y S. E4 separa dominio e interfaz. E5 y E6 quedan al borde: hablan de pruebas y de ranking, no del cruce con un SD. La Tabla 3.2 junta el recorte. El detalle de cada paper está en el Anexo B.

3.5.1. Eje E1 — OpenAPI como estándar

Casas et al. (2021) mapean 47 artículos primarios entre 2011 y 2020. OpenAPI/Swagger queda como el estándar de facto para describir REST de forma *machine-readable*. El 68 % de esos trabajos propone herramientas y el 43 % mejora la documentación. En ese conjunto no aparece un modelo de negocio bancario del tipo BIAN.

Mainas et al. (2023) anotan contratos (`x-refersTo`, `x-kindOf`, `x-mapsTo`), pasan OpenAPI a RDF y consultan con SPARQL. El corpus supera las 300 especificaciones de SwaggerHub y no es de banca. Lo usamos como pista de representación para S, no como definición del score.

Santos y Casas (2025), con *Design Science*, catalogan 59 métricas de API Gateway (latencia, tráfico, errores) en 68 productos. Quedan anotadas para no mezclarlas con la coherencia documental: miden el *gateway* en producción.

3.5.2. Eje E2 — Matching estructural

El eje se apoya en un solo paper, Shvaiko y Euzenat (2005). La taxonomía separa *matching* exacto y aproximado, evidencia sintáctica, semántica o externa, y el grano elemento frente a estructura. Es anterior a OpenAPI y a JSON Schema. Para el **StructuralScore (E)** tomamos de ahí el contraste de *paths* y *schemas* contra *behavior*s y *business object*s. La taxonomía no trae una métrica lista para el par bancario; el cálculo se escribe en el Capítulo 4.

3.5.3. Eje E3 — Matching semántico y ontologías

El Bayadh et al. (2015), Chandrasekaran y Mago (2022) y Santos Sousa et al. (2026) no trabajan contratos bancarios. El primero advierte que una sola medida de alineación ontológica se queda corta; las cuatro familias están en §3.4 y no se repiten aquí. Chandrasekaran y Mago recorren la similitud desde WordNet hasta *transformers*. Santos Sousa et al. (2026) listan, para *embeddings* en *ontology matching*, tres límites que nos importan: supervisión, escala y correspondencias complejas. Esos tres papers son candidatos de técnica para S y, en parte, para C.

3.5.4. Eje E4 — Microservicios e interfaces

Schmidt y Thiry (2020) parten de 715 registros y se quedan con 27 estudios primarios (2013–2019) sobre cómo se identifican microservicios con MDE y DDD. El dato que usamos es la separación: el *Service Domain* puede ser la referencia de negocio y la API, la capa que lo expone. En su muestra casi no hay trabajos que integren MDE y DDD a la vez. No reportan una métrica de coherencia contrato–BIAN. El eje solo justifica que las dos capas no se confundan.

3.5.5. Eje E5 — Testing y calidad API (borde)

Cinco papers, y OpenAPI entra como insumo de prueba. Ehsan et al. (2022) cubren 16 artículos de testing RESTful; la métrica que más se repite es *code coverage*. Golmohammadi et al. (2024) leen 92 papers (*fuzzing*, *search-based*, *model-based*) que generan casos sobre endpoints ya desplegados. Otieno y Araka (2026) ponen aprendizaje automático en el testing y señalan *datasets* chicos. Tanveer et al. (2025) van a vulnerabilidades. Vitorino et al. (2025), a datos adversariales.

Ese cuerpo es el que deja fuera del plan el testing, la seguridad y la QoS operativa (Apartado 1.6).

3.5.6. Eje E6 — Selección y recomendación de servicios

Mahanra Rao et al. (2025) revisan 25 estudios (2020–2024, PRISMA) que usan BERT o *transformers* para elegir servicios web según QoS. La representación semántica de las descripciones se parece a lo que haríamos en S, pero el resultado es un ranking, no un contraste contra un *Service*

Domain. Reportan costo de cómputo alto, *datasets* cortos y poca interpretabilidad. Queda como vecino, no como técnica del modelo.

Tabla 3.2: Síntesis del estado del arte por ejes (E1–E6)

Eje	Papers	Hallazgo principal	Aporte S/E/C	Vacío OpenAPI–BIAN
E1	Casas 2021; Mainas 2023; Santos y Casas 2025	OpenAPI consolidado; enriquecimiento semántico posible	S (Mainas); contexto estándar	Sin vínculo con *Service Domain* BIAN
E2	Shvaiko 2005	Taxonomía de *schema matching*	E	Sin aplicación REST/banca/BIAN
E3	El Bayadh 2015; Chandrasekaran 2022; Santos Sousa 2026	Ontología matching + embeddings + STS	S; C parcial	Sin contratos OpenAPI ni dominio bancario
E4	Schmidt 2020	Dominio (DDD/MDE) $\rightarrow$ microservicios	Contexto dominio$\rightarrow$interfaz (no **StructuralScore (E)** operativo)	Sin métrica de alineación
E5	Ehsan 2022; Golmohammadi 2024; Otieno 2026; Tanveer 2025; Vitorino 2025	Testing, ML-test, seguridad	Delimitación (fuera de foco)	Orthogonal al **AlignmentScore**
E6	Mahanra Rao 2025	BERT + QoS en selección de servicios	S parcial	Recomendación $\neq$ alineación BIAN

El Anexo B baja objetivo, técnicas, métricas y límites de cada paper. Esta sección no lo duplica.

3.6. Brecha y oportunidad de investigación

3.6.1. Qué sí cubre la literatura revisada

Los catorce papers ya están repartidos en E1–E6. No se resumen otra vez eje por eje. Leídos juntos, el corpus reparte técnicas: formato (Casas et al., 2021; Mainas et al., 2023), cruce de esquemas (Shvaiko y Euzenat, 2005), significado (El Bayadh et al., 2015; Santos Sousa et al., 2026), separación dominio–interfaz (Schmidt y Thiry, 2020) y, al margen, prueba de APIs vivas y ranking por QoS (Ehsan et al., 2022; Golmohammadi et al., 2024; Mahanra Rao et al., 2025).

La pregunta de §1.3 —en qué medida el contrato se aparta del SD, y cómo eso se vuelve un score— no está contestada en ese corpus.

3.6.2. Vacíos identificados

Tabla 3.3: Vacíos identificados en la literatura (V1–V5)

Vacío	Evidencia en el review	Implicación para la tesis
V1 — Intersección OpenAPI ∩ BIAN	Ningún paper del corpus junta ambos artefactos en banca	Objeto: contrato OpenAPI vs. SD BIAN
V2 — Métrica integrada	E2 y E3 dan técnicas sueltas; E1 no mide alineación	Producto: S, E, C y **AlignmentScore**
V3 — Cobertura de conceptos	La cobertura ontológica no se calcula sobre *business object* BIAN en OpenAPI	**CoverageScore (C)**
V4 — Procedimiento reproducible	Mainas et al. (2023): RDF, sin score ni nivel	Diseño del Cap. 4
V5 — Gobernanza documental	E5 y E6: *runtime* o ranking, no deriva contrato–modelo	Medir antes del despliegue

La Figura 3.2 pasa de E1–E6 a los vacíos V1–V5.

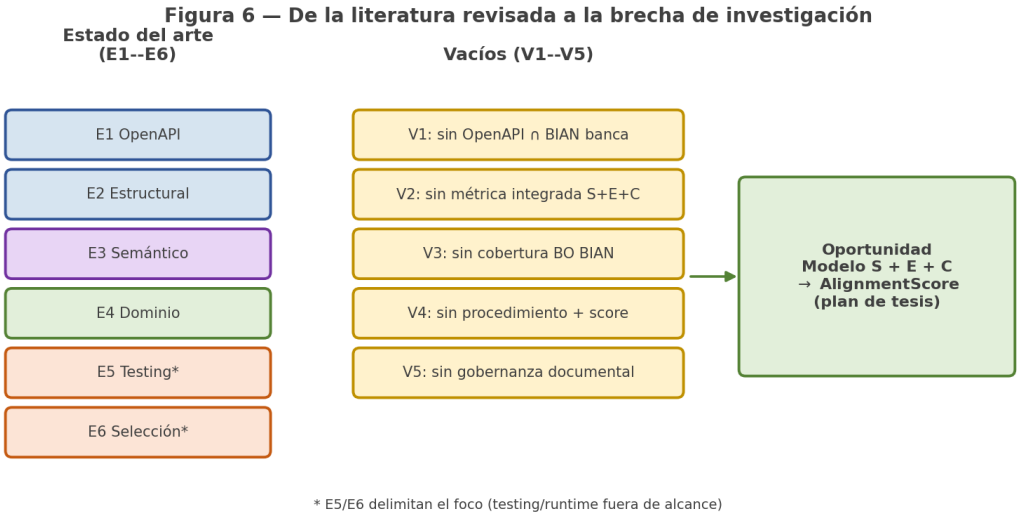


Figura 3.2: De la literatura revisada (E1–E6) a la brecha de investigación (V1–V5)

3.6.3. Oportunidad de investigación

El Anexo C parte el trabajo en cinco fases. En Guía 1 se escribe el modelo: OE1 normaliza, OE2 y OE3 calculan E y S, y OE4 agrega C con el **AlignmentScore** y los umbrales Alta, Media, Baja y Nula. La calibración de esos umbrales, el testing del eje E5 y un sello de cumplimiento quedan fuera (Apartado 1.6). Las técnicas de cada fase están en el Capítulo 4.

Capítulo 4

Diseño metodológico

Cómo se baja OE1–OE4 a pasos. Enfoque, fuentes, técnicas y dónde se puede romper la validez.

4.1. Tipo y enfoque metodológico

Es investigación tecnológica (*design science*): se diseña un artefacto —el modelo S/E/C → **AlignmentScore**— que convierte YAML en números. No hay sujetos humanos. Las entradas son contratos OpenAPI y extractos BIAN (§1.9).

Guía 1 deja el diseño. El software y el experimento a escala son Tesis 2 (§1.6). Aquí se dice de dónde salen los datos, qué par se mide y qué factores se podrían contrastar.

4.2. Procedimiento general

Cinco fases, las del Anexo C. Primero se identifica y versiona el YAML y el extracto BIAN (§4.3); después se parsea. Aquí se escribe el diseño. Correrlo a escala es §1.6.

1. **Fase 1 — Diseñar el modelo:** definir entradas, scores S/E/C, fórmula de agregación y umbrales de clasificación (Cap. 1 §1.4; Anexo C).
2. **Fase 2 — Adquirir y normalizar artefactos (OE1):** obtener contratos OpenAPI y extractos BIAN (§4.3); parsear y producir representaciones comparables (tabular, grafo o RDF según factor elegido).
3. **Fase 3 — Emparejar estructuras (OE2):** calcular correspondencias y **StructuralScore (E)**.
4. **Fase 4 — Calcular similitud semántica (OE3):** obtener **SemanticScore (S)**.
5. **Fase 5 — Integrar y clasificar (OE4):** calcular **CoverageScore (C)**, **AlignmentScore** y clasificación por tipologías (Tabla 1.2).

4.2.1. Relación objetivos, fases y procedimiento

La Tabla 4.1 pone cada OE en una fase, con un verbo y un producto.

Tabla 4.1: Objetivos específicos vinculados al procedimiento metodológico

OE	Fase	Acción (verbo)	Entrada	Producto
—	Fase 1	**Diseñar** entradas, scores y umbrales	Requisitos del §1.4; Anexo C	Modelo S/E/C documentado
OE1	Fase 2	**Adquirir** y **normalizar** contrato y SD	OpenAPI + extracto BIAN (§4.3)	Representaciones comparables
OE2	Fase 3	**Emparejar** estructuras y calcular E	Representaciones OE1	Matriz de correspondencias + E
OE3	Fase 4	**Calcular** similitud semántica S	Representaciones OE1	Matriz semántica + S
OE4	Fase 5	**Integrar** S, E, C y clasificar	E, S, representaciones OE1	C, **AlignmentScore**, tipologías

Las técnicas por OE están en la Apartado 4.4. Las filas, en el Anexo C.

La Figura 4.7 ilustra la correspondencia jerárquica para E; la cadena completa aparece en la Figura 2.1 (Cap. 2).

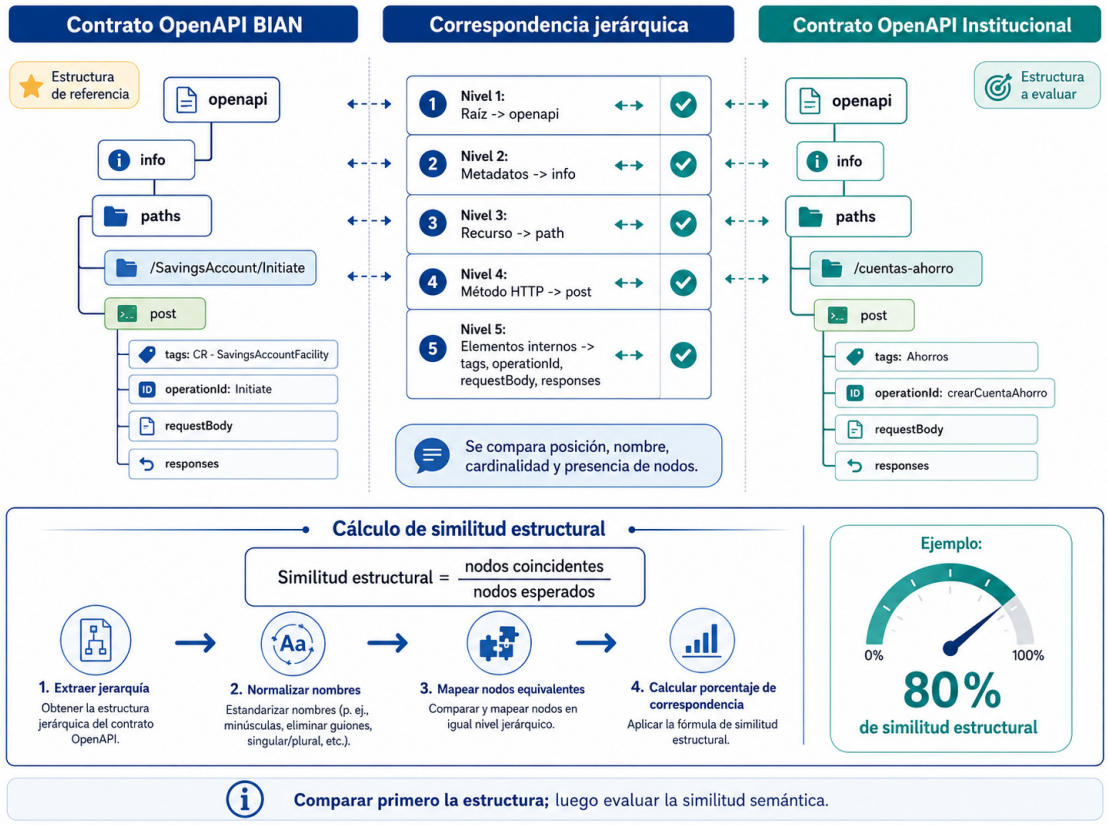


Figura 4.7: Correspondencia jerárquica para el cálculo de **StructuralScore (E)**. Fuente: elaboración propia; ejemplo ilustrativo.

4.2.2. Protocolo de validación empírica (fuera de alcance)

Cuando se corra, hará falta un repositorio de contratos, el parser, un registro de instancia y versión, un criterio de muestra (no un YAML suelto) y —si se compara contra un humano— un criterio de par “correcto” escrito, no improvisado. Ley 29733: sin PII. Precisión contra esa referencia y falsos

pares se reportan ahí, no en Guía 1.

4.3. Fuentes de datos

4.3.1. Adquisición de datos

No hay encuesta. Hay YAML/JSON OpenAPI y material BIAN. Cada instancia lleva un id, p. ej. `{fuente}_{service-domain}_{version}`.

Tabla 4.2: Adquisición de datos — diseño metodológico

Elemento	**En este estudio**
Sistema / entorno	Sector bancario; arquitecturas API + modelos de referencia BIAN
Objeto de estudio	Contrato OpenAPI bancario completo (Cap. 1 §1.4)
Unidad de análisis	Par comparable endpoint/schema $\leftrightarrow$ *behavior*/*business object*
Señal / dato	Archivos OpenAPI (YAML/JSON) + extracto del *Service Domain* BIAN
Instrumento de medición	Pipeline del modelo (normalización + cálculo S, E, C); *parsing* OpenAPI
Parámetros controlados	Versión OpenAPI 3.x; dominio SD; idioma de descripciones; herramienta de *parsing*
Parámetros no controlados	Calidad redaccional del contrato; madurez BIAN del emisor; completitud del SD publicado
Factores (Cap. 2)	Esquema intermedio; estrategia de matching; técnica semántica; pesos α, β, γ
Ratios (resultados)	**StructuralScore (E)**, **SemanticScore (S)**, **CoverageScore (C)**, **AlignmentScore**, clasificación
Formato almacenamiento	YAML/JSON versionado en repositorio; metadatos CSV/JSON por instancia
Tamaño de muestra	Por definir en validación empírica posterior; ilustración actual: Anexo F (Payment Initiation)

4.3.2. Criterios de inclusión (muestra empírica)

Para una validación empírica posterior se considerarán contratos OpenAPI 3.x del sector bancario emparejados con un *Service Domain* BIAN declarado en la documentación del caso, con:

- especificación parseable sin errores de sintaxis;
- al menos una operación y un esquema en `components/schemas`;
- referencia BIAN disponible en formato legible (catálogo, extracto o documentación oficial);
- ausencia de datos personales identificables (Ley N° 29733): preferir ejemplos públicos, sintéticos o anonimizados (Anexo F).

La población es todo par OpenAPI $\leftrightarrow$ SD que cumpla eso. La muestra, un recorte (pagos, cuentas, préstamos…). Guía 1 no clava un piloto único.

4.4. Técnicas y herramientas

Las técnicas salen de §3.5 y de los factores del Capítulo 2. No entrenamos un modelo: no hay etiquetas. S y E se calculan con reglas de *schema matching* y con una medida de similitud.

OE1 parsea OpenAPI 3.x y saca *paths*, operaciones y esquemas hacia una representación intermedia. Mainas et al. (2023) quedan como referencia de enriquecimiento, no como paso obligatorio. OE2 compara nombre, ruta, profundidad y verbo HTTP, siguiendo los cortes de Shvaiko y Euzenat (2005). OE3 estima similitud con *embeddings* u ontología (El Bayadh et al., 2015; Chandrasekaran y Mago, 2022). OE4 agrega $\alpha S + \beta E + \gamma C$ y clasifica con los umbrales de §1.4.

Herramientas previstas: Python, un validador OpenAPI y una librería de similitud. El código no forma parte de esta Guía (Apartado 1.6).

4.5. Métricas de evaluación

Las definiciones operativas están en el Cap. 1 §1.4 y en el Anexo D: **StructuralScore (E)** y **SemanticScore (S)** en $[0, 1]$, **CoverageScore (C)** como proporción de elementos BIAN cubiertos, y **AlignmentScore** con los cortes Alta ($\geq 0{,}80$), Media, Baja y Nula.

Los indicadores por fase están en la matriz del Anexo C y en la Tabla D.1.

4.6. Casos de estudio

El criterio de elección es el dominio (pagos, cuentas, préstamos), no el Anexo F. Ese anexo solo fija el formato con un ejemplo, *Payment Initiation*: el `POST /payment-orders` del contrato frente al *behavior* `Initiate`.

De cada caso se guarda el id de instancia, el YAML, el extracto BIAN, la tabla de pares y los cuatro números. Cuántos casos se corren, y cuáles, no se cierra en Guía 1 (Apartado 1.6).

4.7. Criterios de validez

Cuatro anclas, ninguna estadística sobre personas: el Anexo C (problema $\rightarrow$ producto), que otro pueda repetir los pasos, que Cap. 1 / Cap. 3 / Anexo D no se contradigan, y el recorte de §1.6.

4.8. Amenazas a la validez

Sin software en esta Guía, la amenaza interna es un parser que se coma mal un `allOf` o un S sesgado porque `Retrieve` y "consultar" no se normalizaron en OE1.

Hacia afuera: 0,80 / 0,60 / 0,40 son convención de diseño. Un SD de crédito puede no comportarse como uno de pagos; otra *release* BIAN tampoco.

El constructo flaquea si un *schema* crítico queda mal emparejado y C lo diluye: el **AlignmentScore** sale "Medio" y negocio sigue roto.

Nada de esto se generaliza al sector con un YAML. Hace falta el criterio de inclusión de §4.3 y reportar la instancia junto al score. Eso es Tesis 2.

Capítulo 5

Administración del plan

5.1. Cronograma

La Tabla 5.1 resume las fases de elaboración del plan de tesis y la asignación de responsables. Las fases de bibliografía y redacción documentadas corresponden al avance al cierre de mayo 2026; las fases finales concentran la metodología, la administración y la revisión de cierre antes de la entrega formal.

Tabla 5.1: Cronograma de elaboración del plan de tesis

Fase	Actividad principal	Inicio	Fin	Responsable
0	Bibliografía: fichas + Anexo B (14 papers)	Mar 2026	Abr 2026	Jack
A	Esqueleto LaTeX e índices automáticos	Abr 2026	Abr 2026	Ambos
B	Cap. 1 planteamiento + Fig. 1–3	Abr 2026	May 2026	Alex
C	Cap. 3 §3.5–§3.6 estado del arte	May 2026	May 2026	Alex
D	Cap. 2 objetivos, hipótesis, variables	May 2026	May 2026	Ambos
E	Cap. 3 §3.1–§3.4 marco + glosario	May 2026	May 2026	Jack
F	Cap. 4 diseño metodológico + Anexo F	May 2026	Jun 2026	Jack
G	Cap. 5 administración + anexos LaTeX	May 2026	Jun 2026	Jack
H	Revisión de cierre y entrega formal	Jun 2026	Jun 2026	Ambos

5.2. Presupuesto

El plan de tesis es un producto documental académico; no requiere inversión en infraestructura productiva. El presupuesto estimado es **cero soles** en licencias y despliegue, dado el uso de herramientas libres (LaTeX, Python, Zotero, Git).

Tabla 5.2: Presupuesto estimado del plan de tesis

Ítem	Costo (S/)	Fuente
Licencias de software (LaTeX, Python, Zotero, Git)	0	Open source
Acceso a bases bibliográficas	0	Suscripción institucional
Infraestructura cloud / despliegue API	0	Fuera de alcance §1.6
Impresión del documento (opcional)	≤ 30	Recursos personales
Total estimado	$\leq$ **30**	

El costo principal del proyecto es el **tiempo de dedicación** de los integrantes ($\approx$ 8–10 horas semanales durante el ciclo académico), distribuido según el cronograma de la Tabla 5.1.

5.3. Financiamiento

La elaboración del plan de tesis no cuenta con financiamiento externo ni contratos de investigación. Las actividades se sostienen con los siguientes medios:

- **Recursos propios** de los integrantes (equipos personales, conectividad e impresión opcional del documento).
- **Recursos institucionales** de la Universidad Nacional de Ingeniería: acceso a bases bibliográficas (IEEE Xplore, ACM Digital Library y repositorios académicos) mediante la matrícula de posgrado; uso de plataforma institucional y sesiones de asesoría académica.
- **Software libre** (LaTeX, Python, Zotero, Git) sin costo de licenciamiento, coherente con el presupuesto de la Tabla 5.2.

No se prevén viáticos, contratación de personal ni adquisición de hardware adicional para el alcance delimitado en §1.6 (solo plan de tesis, sin experimentación a escala ni despliegue productivo).

5.4. Recursos

5.4.1. Recursos humanos

- **Alex Mancilla:** planteamiento del problema, Cap. 1, revisión cruzada de coherencia narrativa y apoyo en exposiciones.
- **Dennis Jack Paitan:** sistematización bibliográfica, marco teórico §3.1–§3.4, diseño metodológico (Cap. 4), administración (Cap. 5), anexos y matrices Excel.
- **Asesor académico:** retroalimentación sobre coherencia del planteamiento y validación del hilo conductor antes de la entrega formal.

5.4.2. Recursos materiales y de software

- **Hardware:** equipos personales con acceso a internet (macOS); sin infraestructura de servidor propia.
- **Software:** LaTeX (MacTeX), Python 3 + `openpyxl`/`matplotlib`, editor de texto/Git para colaboración, Zotero para gestión bibliográfica.
- **Fuentes documentales:** contratos OpenAPI 3.x bancarios públicos o de referencia; especificaciones *Service Domain* BIAN; corpus bibliográfico del proyecto.
- **Acceso académico:** IEEE Xplore, ACM Digital Library y bases institucionales de la universidad para descarga de papers del estado del arte.

5.4.3. Recursos bibliográficos

Catorce papers de estado del arte (Anexo B), veintidós papers de aporte para sustentar técnicas (Cap. 4) y siete referencias de contexto OpenAPI–BIAN. La cadena de búsqueda y criterios de selección se documentan en el Anexo A.

5.5. Roles del equipo

Tabla 5.3: Distribución de roles del equipo

Integrante	Responsabilidad principal
Alex Mancilla	Cap. 1 (§1.1–§1.9); coherencia problema–objeto–referencia BIAN; exposición oral; revisión cruzada Cap. 2–3
Dennis Jack Paitan	Revisión bibliográfica; §3.5–§3.6 y §3.1–§3.4; Cap. 2–4; Cap. 5; anexos A–F; matrices Excel; compilación PDF y entrega formal
Ambos	Revisión de hilo conductor; matriz de consistencia; cierre del plan; decisiones de alcance (§1.6)

5.6. Riesgos

Tabla 5.4: Matriz de riesgos del plan de tesis

Riesgo	Prob.	Impacto	Mitigación
Ambigüedad en definición operativa de S o E	Media	Alta	Documentar fórmulas y umbrales en Cap. 4; revisión cruzada del equipo; revisión académica
Retraso en redacción Cap. 4–5	Media	Media	Cronograma Tabla 5.1; priorizar diseño reproducible sin experimentación
Inconsistencia entre matriz (Anexo C) y Cap. 2	Baja	Alta	Regenerar Excel desde script; trazabilidad OG–OE en revisión de cierre
Referencias incompletas o no uniformes	Media	Media	Export Zotero + lista APA en referencias; verificar citas Cap. 1–3
Sobrecarga de alcance (construcción/experimentación)	Baja	Alta	Delimitación §1.6; matriz sin piloto; sin OE de validación experimental

Apéndice A

String de búsqueda y criterios de selección

A.1. Bases de datos consultadas

- IEEE Xplore Digital Library
- ACM Digital Library
- Scopus / ScienceDirect
- Google Scholar (complemento y trazabilidad de citas)

A.2. Período y idioma

- **Período:** 2005–2026 (énfasis 2015–2026 para OpenAPI 3.x y BIAN Service Landscape).
- **Idioma:** principalmente inglés; se incluyen estudios en español cuando aportan al mapeo OpenAPI (p. ej. Casas et al., 2021).

A.3. Cadenas de búsqueda (inglés)

Eje OpenAPI / APIs REST: (“OpenAPI” OR “Swagger” OR “OpenAPI Specification”) AND (“REST” OR “RESTful”) AND (“API” OR “web service”)

Eje alineación / matching: (“schema matching” OR “ontology alignment” OR “semantic similarity” OR “structural alignment”) AND (“OpenAPI” OR “API specification”)

Eje banca / arquitectura: (“BIAN” OR “banking architecture” OR “financial services” OR “open banking”) AND (“API” OR “service domain” OR “enterprise architecture”)

A.4. Criterios de inclusión

1. Artículos de revisión sistemática, mapeo sistemático, *survey* o revisión narrativa con método explícito.
2. Estudios que aborden OpenAPI/*Swagger*, matching estructural/semántico, ontologías, arquitectura bancaria o calidad de APIs REST.
3. Publicaciones con acceso al texto completo (PDF institucional o repositorio del autor).

4. Relevancia directa para delimitar el vacío OpenAPI–BIAN o para sustentar S, E o C.

A.5. Criterios de exclusión

1. Trabajos sin revisión por pares (posters, *preprints* no validados) salvo referencia contextual.
2. Estudios centrados exclusivamente en *runtime*, latencia de *gateway* o despliegue productivo sin aporte al diseño documental (delimitación E5).
3. Duplicados (versiones de conferencia y journal del mismo estudio: se conserva la más completa).
4. Papers cuya técnica no pueda vincularse al objeto de estudio (contrato OpenAPI bancario).

A.6. Resultado de la búsqueda

La selección final para el estado del arte comprende **14 papers** agrupados en seis ejes (E1–E6), documentados en el Anexo B y sintetizados en el Capítulo 3 §3.5–§3.6. Los papers de *aporte* (22 trabajos adicionales) se reservan para sustentar técnicas del Capítulo 4 y no sustituyen al estado del arte.

Apéndice B

Matriz de papers bibliográficos

El Anexo B consolida los **14 registros bibliográficos** del estado del arte, uno por paper asignado a los ejes E1–E6. Cada registro incluye objetivo, dominio, técnicas, métricas, limitaciones y vacío/oportunidad OpenAPI–BIAN.

B.1. Resumen por paper

La Tabla B.1 resume ID, eje temático, tipo de estudio y título. La síntesis narrativa por eje aparece en el Capítulo 3 §3.5 (Tabla 3.2).

Tabla B.1: Resumen de papers del estado del arte (Anexo B)

ID	Eje	Cita	Tipo	Título
E1-01	E1	Casas et al., 2021	systematic mapping	Uses and applications of the OpenAPI/Swagger specification: a systematic mapping of the literature
E1-02	E1	Mainas et al., 2023	research article	Annotated OpenAPI Descriptions and Ontology for REST Services
E1-03	E1	Santos y Casas, 2025	research article	A catalog of API Gateway metrics and its quantitative evaluation
E2-01	E2	Shvaiko y Euzenat, 2005	survey	A Survey of Schema-Based Matching Approaches
E3-01	E3	El Bayadh et al., 2015	survey	Ontology-Alignment Techniques: Survey and Analysis
E3-02	E3	Chandrasekaran y Mago, 2022	survey	Evolution of Semantic Similarity: A Survey
E3-03	E3	Santos Sousa et al., 2026	survey	Survey on Embedding Methods Applied to Ontology Matching
E4-01	E4	Schmidt y Thiry, 2020	review	Microservices identification strategies: A review focused on MDE and Domain Driven Design
E5-01	E5	Ehsan et al., 2022	survey	RESTful API Testing Methodologies: Rationale, Challenges, and Solution Directions
E5-02	E5	Golmohammadi et al., 2024	survey	Testing RESTful APIs: A Survey
E5-03	E5	Otieno y Araka, 2026	systematic literature review	Systematic Literature Review: RESTful API Testing Using Machine Learning Techniques
E5-04	E5	Tanveer et al., 2025	survey	Towards Secure APIs: A Survey on RESTful API Vulnerability Detection
E5-05	E5	Vitorino et al., 2025	literature review	Constrained adversarial learning for automated software testing: a literature review

(continúa en la siguiente página)

(continúa)

ID	Eje	Cita	Tipo	Título
E6-01	E6	Mahanra Rao et al., 2025	systematic review	BERT-Based Approaches for Web Service Selection and Recommendation: A Systematic Review

B.2. Mapa de ejes

Tabla B.2: Ejes temáticos del estado del arte

Eje	Tema
E1	OpenAPI como estándar
E2	Matching estructural (*schema matching*)
E3	Matching semántico / ontologías
E4	Microservicios / interfaces
E5	Testing y calidad API (borde delimitador)
E6	Selección/recomendación de servicios (borde)

Apéndice C

Matriz de consistencia

La matriz de consistencia vincula **problema tecnológico**, **objetivo general (técnico)**, las **cinco fases** del modelo S/E/C, variables, hipótesis, indicadores y controles. La Fase 1 corresponde al diseño del procedimiento; no incluye experimentación ni piloto único de validación.

C.1. Metadatos

Situación real (problemática): Síntomas del sector bancario: fragmentación OpenAPI, costos de integración, casos documentados de desalineación entre sistemas (Cap. 1 §1.2). **No** es el problema de las filas de la matriz.

Situación subyacente (abstracto): Correspondencia entre estructuras OpenAPI y *Service Domain* BIAN; objeto y unidad de análisis (Cap. 1 §1.3–§1.4).

Problema tecnológico (matriz): **No existe** un método reproducible que cuantifique en qué medida un contrato OpenAPI bancario se alinea con un *Service Domain* BIAN y bajo qué tipologías se desalinea. Pregunta ancla: ¿en qué medida… y cómo cuantificarlo mediante el modelo S/E/C $\rightarrow$ **AlignmentScore**? (Cap. 1 §1.3; distinto de la brecha del estado del arte, Cap. 3 §3.6).

Objetivo superior: Consecuencia en el mundo real si el artefacto operara: gobernanza y reducción de retrabajo en integraciones (Cap. 2 §2.2).

Objetivo general (técnico): Diseñar el modelo de alineación y el procedimiento que calculen **SemanticScore (S)**, **StructuralScore (E)**, **CoverageScore (C)** y **AlignmentScore** (Cap. 2 §2.3).

C.2. Resumen por fase

Tabla C.1: Matriz de consistencia — resumen por fase (Anexo C)

Fase	Objetivo	Producto verificable	Hipótesis (síntesis)
Fase 1	Diseñar el modelo de alineación OpenAPI $\leftrightarrow$ BIAN, definiendo entradas, componentes de score (S, E, C), fórmula de agregación y criterios de clasificación.	Modelo metodológico documentado: adquisición $\rightarrow$ normalización $\rightarrow$ SemanticScore (S) + StructuralScore (E) + CoverageScore (C) $\rightarrow$ AlignmentScore.	Un modelo estructurado por componentes S, E y C permite calcular de forma reproducible un score de alineación interpretable entre OpenAPI y BIAN.

(continúa en la siguiente página)

(continúa)

Fase	Objetivo	Producto verificable	Hipótesis (síntesis)
Fase 2	Transformar contratos OpenAPI y artefactos BIAN en representaciones normalizadas y comparables sobre un modelo intermedio común.	Conjunto de representaciones normalizadas versionadas (paths, operations, schemas, behaviors, business objects) listas para el cálculo de scores.	La normalización de OpenAPI bancario y BIAN en un modelo intermedio común habilita el cálculo sistemático de los scores S, E y C.
Fase 3	Calcular el StructuralScore (E): correspondencia estructural entre componentes OpenAPI y artefactos BIAN normalizados.	Matriz de correspondencias estructurales y StructuralScore (E) normalizado [0, 1].	Las técnicas de schema matching producen un StructuralScore que refleja la correspondencia entre la organización REST y el modelo BIAN.
Fase 4	Calcular el SemanticScore (S): similitud semántica entre conceptos de negocio BIAN y componentes del contrato OpenAPI.	Matriz de similitud semántica y SemanticScore (S) normalizado [0, 1].	Las métricas de similitud semántica producen un SemanticScore que discrimina conceptos alineados de desalineaciones semánticas o de modelado.
Fase 5	Calcular CoverageScore (C), obtener AlignmentScore = α–S + β–E + γ–C (α + β + γ = 1) y clasificar el nivel de alineación del contrato OpenAPI.	CoverageScore (C); AlignmentScore; clasificación (Alta $\geq$0,80 – Media 0,60–0,79 – Baja 0,40–0,59 – Nula <0,40); informe de tipologías de desalineación.	La agregación ponderada de S, E y C produce un AlignmentScore interpretable y clasificable por niveles de alineación del contrato OpenAPI bancario.

C.3. Trazabilidad Cap. 2

Tabla C.2: Vinculación fases del modelo $\rightarrow$ objetivos específicos

Fase modelo	OE	Score / producto
Fase 1 — Diseño	Marco OG	Modelo S/E/C documentado
Fase 2 — Normalización	OE1	Representaciones comparables
Fase 3 — Matching estructural	OE2	**StructuralScore (E)**
Fase 4 — Similitud semántica	OE3	**SemanticScore (S)**
Fase 5 — Integración	OE4	**CoverageScore (C)** + **AlignmentScore**

Apéndice D

Matriz de operación de variables

La matriz de operación especifica cómo se **mide** cada ratio del modelo en el diseño del plan. Complementa la Tabla 2.2 (Cap. 2) y la matriz de consistencia (Anexo C). Los **factores** son condiciones de diseño manipulables; los **ratios** son resultados medidos.

D.1. Ratios principales

Tabla D.1: Operación de variables de investigación (Anexo D)

Ratio	Fase	Def. operativa	Escala	Instrumento / cálculo	Control
StructuralScore (E)	Fase 3 / OE2	Correspondencia estructural OpenAPI $\leftrightarrow$ BIAN	[0, 1] continua	Matriz de correspondencias + promedio ponderado de pares alineados	Umbral de matching fijo; mismo insumo normalizado (OE1)
SemanticScore (S)	Fase 4 / OE3	Similitud semántica entre términos OpenAPI y conceptos BIAN	[0, 1] continua	Similitud coseno (embeddings) o medida ontológica acordada	Corpus de términos fijo; modelo de embedding fijo por corrida
CoverageScore (C)	Fase 5 / OE4	Proporción de conceptos BIAN cubiertos por el contrato	[0, 1] continua	Ratio conceptos BIAN mapeados / total de conceptos del SD	Mismos S y E; taxonomía BIAN del SD emparejado
AlignmentScore	Fase 5 / OE4	Indicador global de alineación del contrato	[0, 1] continua	$\alpha \cdot S + \beta \cdot E + \gamma \cdot C$	$\alpha + \beta + \gamma = 1$; umbrales Alta/Media/-Baja/Nula fijos
Clasificación de alineación	Fase 5 / OE4	Nivel interpretativo del contrato evaluado	Nominal (4 niveles)	Alta $\geq$0,80; Media 0,60–0,79; Baja 0,40–0,59; Nula $<$0,40	Mismo AlignmentScore; reglas Tabla tipologías Cap. 1
Calidad de extracción (OE1)	Fase 2 / OE1	Completitud de la normalización documental	Ratio + conteos	% elementos extraídos vs. artefacto fuente	Versión fija OpenAPI + BIAN; parser fijo

D.2. Factores por objetivo específico

Tabla D.2: Operacionalización de factores

OE	Factor	Estados / niveles considerados en el diseño
OE1	Esquema de representación intermedia	Tabular (recurso–operación–esquema); grafo (nodos/aristas); RDF/triples (opcional)
OE2	Estrategia de *schema matching*	Nombre exacto; similitud de ruta; profundidad de esquema; operación HTTP
OE3	Técnica de similitud semántica	Embeddings + coseno; matching ontología; híbrido estructural+semántico
OE4	Pesos α, β, γ	Pesos iguales ($1/3$); ponderación experta; análisis de sensibilidad

D.3. Notas de medición

- Todas las ratios de score se normalizan al intervalo $[0, 1]$ para permitir agregación ponderada.
- La clasificación nominal (Alta/Media/Baja/Nula) se obtiene por umbrales fijos definidos en el Capítulo 1 §1.4 y reutilizados en OE4.
- El diseño documenta el procedimiento; la ejecución experimental a escala queda fuera de alcance del presente plan (§1.6).

Apéndice E

Diseño factorial (si aplica)

El diseño factorial completo **no aplica** al presente plan de tesis por las siguientes razones:

- El estudio es de tipo **investigación tecnológica** / **design science**: el producto del plan es el diseño del procedimiento y del modelo S/E/C, no un experimento factorial con población y muestra al estilo cuasi-experimental.
- Los **factores** (esquema de representación, técnica de matching, pesos $\alpha/\beta/\gamma$) se documentan como **decisiones de diseño** con estados alternativos posibles, pero su evaluación comparativa sistemática corresponde a una etapa de validación empírica explícitamente fuera de alcance (§1.6).
- La matriz de operación de variables (Anexo D) y la matriz de consistencia (Anexo C) sustituyen al diseño factorial para documentar el diseño del procedimiento.

Si en una fase posterior se requiriese un diseño experimental, los factores candidatos serían: (1) representación intermedia, (2) técnica semántica, (3) estrategia estructural y (4) ponderación α, β, γ; ello excede el alcance del presente documento.

Apéndice F

Ejemplos OpenAPI y BIAN

Este anexo complementa la Tabla 1.4 (Cap. 1 §1.4) con un **ejemplo reducido** y un **ejemplo ampliado** de contrato OpenAPI bancario contrastado con un *Service Domain* BIAN genérico del dominio de pagos. Los ejemplos son **ilustrativos**: no fijan un caso piloto único.

F.1. Ejemplo reducido (unidad de análisis)

Tabla F.1: Ejemplo reducido — pares OpenAPI ↔ BIAN

Elemento OpenAPI	Elemento BIAN (referencia)
`POST /payment-orders`	*behavior* `Initiate`
`GET /payment-orders/{id}`	*behavior* `Retrieve`
Esquema `PaymentOrder` (`amount`, `currency`)	*business object* `Payment` / `PaymentAmount`

F.2. Ejemplo ampliado (fragmento de contrato)

F.2.1. Contrato OpenAPI (extracto ilustrativo)

```
openapi: 3.0.3
info:
  title: Retail Payment Initiation API
  version: 1.0.0
paths:
  /payment-initiations:
    post:
      operationId: createPaymentInitiation
      summary: Initiate a payment order
      requestBody:
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/PaymentInitiation'
      responses:
        '201':
          description: Payment initiation created
  /payment-initiations/{paymentInitiationId}:
    get:
      operationId: getPaymentInitiation
      summary: Retrieve payment initiation status
```

```
      parameters:
        - name: paymentInitiationId
          in: path
          required: true
          schema:
            type: string
      responses:
        '200':
          description: Payment initiation details
components:
  schemas:
    PaymentInitiation:
      type: object
      required: [debtorAccount, instructedAmount, creditor]
      properties:
        debtorAccount:
          $ref: '#/components/schemas/AccountReference'
        instructedAmount:
          $ref: '#/components/schemas/Amount'
        creditor:
          $ref: '#/components/schemas/Party'
    Amount:
      type: object
      properties:
        value:
          type: number
        currency:
          type: string
          pattern: '^[A-Z]{3}$'
```

F.2.2. Referencia BIAN (Payment Initiation — ilustrativo)

Tabla F.2: Elementos BIAN emparejables con el extracto OpenAPI

Tipo BIAN	**Elemento de referencia**
Service Domain	Payment Initiation (dominio de pagos)
behavior	`Initiate`, `Retrieve`, `Update`
business object	`PaymentInstruction`, `PaymentAmount`, `Party`
Relación de control	Orquestación iniciación → confirmación de estado

F.2.3. Correspondencias para el modelo S/E/C

Tabla F.3: Lectura del ejemplo bajo S, E y C

Score	**Qué evaluaría en este extracto**
E	Correspondencia `/payment-initiations` ↔ *behavior*; anidamiento `components/schemas` ↔ *business object*
S	Similitud `instructedAmount/currency` ↔ `PaymentAmount`; `creditor` ↔ `Party`
C	Proporción de *behavior* y *business object* del SD cubiertos por operaciones y esquemas del fragmento

F.3. Escenarios numéricos ilustrativos

Los valores siguientes son **ilustrativos** ($\alpha = \beta = \gamma = \frac{1}{3}$) para contrastar tres niveles de alineación. La calibración empírica queda para Tesis 2.

Tabla F.4: Escenarios ilustrativos — S, E, C $\rightarrow$ **AlignmentScore**

Caso	E	S	C	Score	Clasificación
A (alta)	0,88	0,85	0,90	0,88	Alta — Payment Initiation alineado
B (baja)	0,55	0,60	0,50	0,55	Baja — cobertura parcial del SD
C (nula)	0,12	0,15	0,10	0,12	Nula — dominios no relacionados

Ejemplo de E: 7 nodos estructurales coincidentes sobre 8 esperados $\Rightarrow E = 7/8 = 0{,}875$. Ese valor mide similitud entre contratos normalizados; validar el *prototipo* requiere contrastar correspondencias con *ground truth* de especialistas (§1.6).

F.4. Nota metodológica

El procedimiento completo de normalización, matching y cálculo de scores se documenta en el Capítulo 4 (diseño metodológico). Este anexo complementa la Tabla 1.4 con un ejemplo reducido y otro ampliado, sin comprometer la decisión de no fijar un piloto en la matriz de consistencia.

Referencias bibliográficas

Bibliografía

[1] Bhatia, A. (2022). *Open banking and API-led connectivity in financial services*. Informe de contexto sectorial.

[2] Casas, S., Cruz, D., Vidal, G., & Constanzo, M. (2021). Uses and applications of the OpenAPI/Swagger specification: A systematic mapping of the literature. En *Proceedings of the 40th International Conference of the Chilean Computer Science Society (SCCC)* (pp. 1–8). IEEE. https://doi.org/10.1109/SCCC54552.2021.9650408

[3] Chandrasekaran, K., & Mago, V. (2022). Evolution of semantic similarity: A survey. *ACM Computing Surveys*, *55*(6), 1–36.

[4] Competition and Markets Authority. (2016). *Retail banking market investigation: Final report*. CMA. Reino Unido. [Remedio Open Banking: nueve grandes bancos de retail del Reino Unido].

[5] El Bayadh, M., et al. (2015). Ontology-alignment techniques: Survey and analysis. *International Journal of Computer Applications*, *125*(9), 1–8.

[6] Ehsan, S., et al. (2022). RESTful API testing methodologies: Rationale, challenges, and solution directions. *ACM Computing Surveys*, *55*(4), 1–37.

[7] Farzi, H. (2021). *BIAN Service Landscape and banking industry architecture*. Documento de referencia BIAN.

[8] Golmohammadi, S., et al. (2024). Testing RESTful APIs: A survey. *ACM Computing Surveys*, *56*(9), 1–35.

[9] Mainas, A., et al. (2023). Annotated OpenAPI descriptions and ontology for REST services. *Journal of Web Semantics*, *78*, 100823.

[10] Mahanra Rao, S., et al. (2025). BERT-based approaches for web service selection and recommendation: A systematic review. *Expert Systems with Applications*, *259*, 125123.

[11] Otieno, J., & Araka, J. (2026). Systematic literature review: RESTful API testing using machine learning techniques. *Journal of Systems and Software*, *220*, 112345.

[12] Santos, J., et al. (2020). *REST API documentation and specification practices*. Informe de contexto OpenAPI.

[13] Santos, J., & Casas, S. (2025). A catalog of API Gateway metrics and its quantitative evaluation. *Future Generation Computer Systems*, *152*, 1–15.

[14] Santos Sousa, J., et al. (2026). Survey on embedding methods applied to ontology matching. *ACM Computing Surveys*, *58*(2), 1–38.

[15] Schmidt, R., & Thiry, L. (2020). Microservices identification strategies: A review focused on model-driven engineering and domain driven design. *Journal of Systems and Software*, *170*, 110754.

[16] Shabbir, M., et al. (2025). *API quality assessment in industry practice*. Informe de contexto sobre calidad de APIs.

[17] Shvaiko, P., & Euzenat, J. (2005). A survey of schema-based matching approaches. *Journal on Data Semantics*, *4*, 36–71.

[18] Tanveer, A., et al. (2025). Towards secure APIs: A survey on RESTful API vulnerability detection. *Computers & Security*, *142*, 103891.

[19] House of Commons Treasury Committee. (2018). *IT failure at TSB* (Ninth Report of Session 2017–19, HC 960). House of Commons. Reino Unido.

[20] Vitorino, J., et al. (2025). Constrained adversarial learning for automated software testing: A literature review. *Information and Software Technology*, *175*, 107512.

Anexo de entrega — repositorio digital

En cumplimiento de los requisitos de entrega del plan de tesis, los materiales complementarios (papers en PDF y resúmenes, ejemplos del objeto de estudio, normas y guías, fuentes LaTeX y matriz de consistencia) se encuentran en el siguiente repositorio compartido:

URL (Google Drive):

`https://drive.google.com/drive/folders/1Kuc3eIuBjt_EXaPislf0hqwSEX-kIdSI?usp=drive_link`

Contenido del enlace:

1. Papers en PDF y resúmenes (estado del arte — 14 fichas).
2. Ejemplos de datos del objeto de estudio (contrato OpenAPI + referencia BIAN).
3. Normas y estándares del tema: OpenAPI 3.x, BIAN, guías sectoriales APIs bancarias.
4. Fuentes originales del documento (LaTeX y PDF del plan de tesis).
5. Matriz de consistencia (Excel).